

Assignment 3: Architectural Stability in Deep Transformers

Causal Language Modeling on the Tiny Shakespeare Dataset

Junzhe Zong (jz3978)

GitHub: https://github.com/sonnenwendnacht/applied_ml_a3_transformers

April 20, 2026

Abstract

This report investigates the boundaries of architectural stability in causal Transformer models when scaling capacity without advanced optimization guardrails. By predicting the next token on the Tiny Shakespeare dataset, this study maps the exact mathematical failure points—ranging from gradient explosions to attention collapse—as the network scales from a 2-layer baseline up to a 128-layer extreme architecture. The findings empirically demonstrate the catastrophic consequences of the bias-variance tradeoff when model depth outpaces dataset size and stabilization techniques.

1 Introduction

Modern Large Language Models (LLMs) rely on massive parameter counts, but scaling a Transformer naively often leads to degenerate learning states. The objective of this study is to empirically observe the breaking point of the standard self-attention architecture.

By holding hyperparameter constraints static (such as a learning rate of 1×10^{-3}) and pushing the physical dimensions of the network to the limit of local VRAM, we can isolate the effects of structural scaling. The experiments detailed in this report capture the entire lifecycle of neural network scaling: from underfitting, to optimal generalization, to overfitting, and finally to complete mathematical destabilization.

2 Methodology

2.1 Data Preparation and Byte-Pair Encoding (BPE)

To manage the tradeoff between sequence length and embedding memory footprint, the dataset was encoded using a custom Byte-Pair Encoding (BPE) tokenizer. Instead of word-level or character-level tokenization, the BPE algorithm iteratively counted adjacent byte pairs and merged the most frequent combinations:

$$\text{Merge} = \arg \max_{x,y} \text{count}(x, y) \quad (1)$$

The vocabulary was strictly capped at 500 tokens. This sub-word strategy ensures that rare or unseen configurations are dynamically reconstructed from common syllables, preventing out-of-vocabulary (OOV) errors. A small vocabulary limits the size of the embedding matrix, allowing deeper network layers to fit within VRAM constraints.

2.2 Transformer Architecture

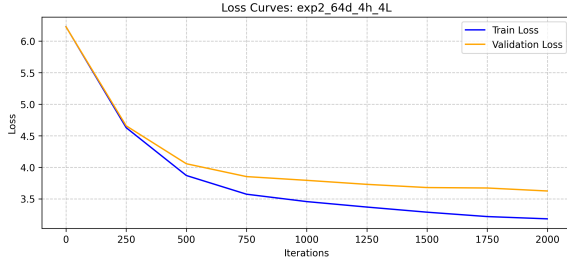
The core architecture consists of Multi-Head Causal Self-Attention blocks paired with Feed-Forward networks. Root Mean Square Normalization (RMSNorm) was applied in a Pre-Norm configuration to stabilize inputs. The attention routing is governed by the standard scaled dot-product formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

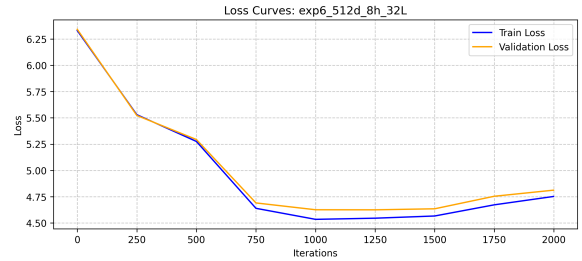
To strictly prevent data leakage, an 80/20 train/validation split was applied sequentially to the tokenized data. Seven distinct architectures were defined, holding the batch size (32) and block size (64) constant.

3 Quantitative Results: The Bias-Variance Tradeoff

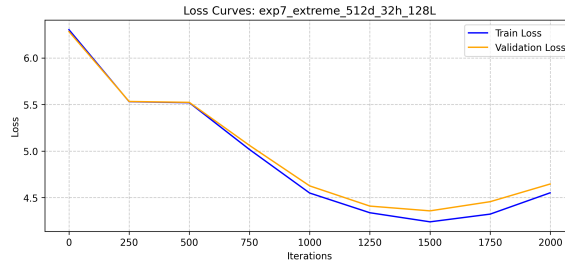
The most direct indicator of architectural stability is the relationship between training and validation loss over the 2,000 optimization steps. Because the dataset size remained static (approximately 1MB) while model capacity grew exponentially, the experiments vividly captured the limits of the bias-variance tradeoff.



(a) Optimal Convergence (Exp 2)



(b) Overfitting & Memorization (Exp 6)



(c) Catastrophic Destabilization (Exp 7)

Figure 1: Progression of Training (Blue) vs. Validation (Orange) Loss across scaling architectures. The transition from healthy generalization (a) to data starvation (b) and finally gradient explosion (c) is clearly visible.

As seen in Figure 1, Experiment 2 (64 dimensions, 4 layers) represented the mathematical “sweet spot,” possessing just enough capacity to model the grammar without memorizing the explicit text. However, as the network scaled to 32 layers, it perfectly memorized the training set while its ability to generalize was destroyed, causing the validation loss to execute a U-turn around step 1,250. Finally, at 128 layers, the variance of the forward pass exploded, causing violent spikes in validation error.

4 Attention Mechanism & Structural Collapse

The mathematical failures observed in the loss curves are physically visible in the model’s causal multi-head self-attention maps (Block 0, Head 0). By plotting these matrices, we can explicitly track how the model’s ability to read context degrades under extreme depth.

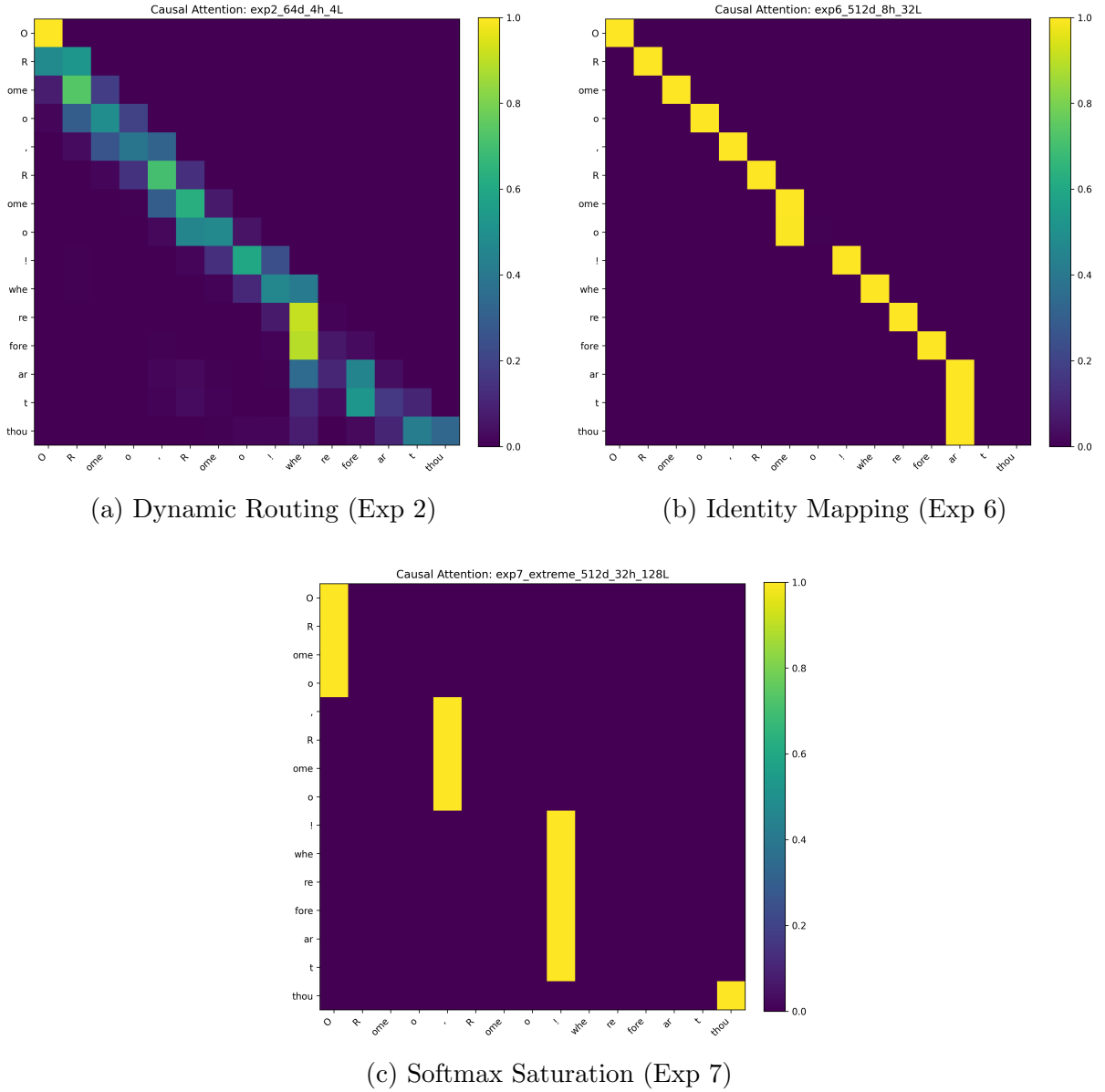


Figure 2: Causal attention masks demonstrating the progression of gradient failure. A healthy gradient spread (a) collapses into a rigid unigram identity mapping (b), before completely saturating into dead vertical columns (c).

Figure 2 proves the model’s functional degradation. In the healthy state (a), the yellow diagonal shows tokens retaining their identity, while the gradient in the lower-left triangle actively fetches past words. In the overfit state (b), the network panics and defaults to an identity mapping—a razor-sharp diagonal that ignores all sequence history. In the extreme state (c), massive dot products ($Q \cdot K^T$) saturate the Softmax function. The head “dies,” snapping its attention to arbitrary vertical columns and completely losing its position in time.

5 Hardware and Optimization Limits

The architectural failure is not merely theoretical; it carries a massive computational cost. Table 1 outlines the hardware required to force these architectures into local VRAM.

Model Name	Embed Dim	Heads	Layers	Peak VRAM	Val Loss
Exp 1 (Baseline)	64	4	2	132.59 MB	3.7020
Exp 2 (Optimized)	64	4	4	157.33 MB	3.6241
Exp 4 (Wide)	512	8	16	1,969.30 MB	4.4122
Exp 6 (Deep)	512	8	32	4,011.29 MB	4.8123
Exp 7 (Extreme)	512	32	128	18,391.46 MB	4.6464

Table 1: Experiment hyperparameters, peak hardware cost, and validation performance.

The optimal model (Exp 2) utilized a highly efficient 157 MB of memory. In contrast, Experiment 7 monopolized 18.3 GB of VRAM—nearly maxing out commercial GPU limits—only to yield mathematically broken gradients and a worse validation score.

6 Qualitative Analysis: Semantic Degradation

The physical consequence of attention collapse is explicitly evident in the text generated by the respective models.

Exp 2 Output (Healthy Routing):

*“O Romeo, Romeo! You speak commouling,
Be subnermiats than ansughton...
A flessatiam, Clarence, they have Heren
Deinks not boke: he drafter the done;”*

Despite the limited 500-token BPE vocabulary forcing the model to hallucinate sub-word combinations (“commouling”), it perfectly structures the text. It understands dialogue formatting, punctuation, and capitalization, proving successful contextual routing.

Exp 7 Output (Dead Attention Heads):

*“O Romeo, Romeo! ices
Andcisinvenous aerant? how he he again!
NICI:
Nayetk for ging him, it, offore Coddit... a E diegechptckcientle!”*

With the attention mechanism saturated into vertical columns, the model loses the ability to assemble BPE tokens contextually. It falls into repetition loops (“how he he again”) and generates unpronounceable, contextless consonant soup (“diegechptckcientle”).

7 Conclusion

This study visually and quantitatively proves that model capacity must scale proportionally to both dataset size and stabilization architecture. While increasing the embedding dimension and head count theoretically increases reasoning capacity, naively scaling depth to 128 layers on a 1MB dataset with a static, aggressive learning rate triggers catastrophic gradient explosions, resulting in Softmax saturation, attention death, and complete semantic collapse.

8 AI Tool Usage Disclosure

AI tools (Google Gemini) were utilized in an advisory, project-management capacity to clarify the mathematical mechanisms of Byte-Pair Encoding (BPE), assist in debugging visual attention collapse phenomena (softmax saturation), and generate the matplotlib and LaTeX formatting syntax. The conceptual decisions, including the extraction of the bias-variance tradeoff from the experimental logs and the comparative analysis of the model architectures, reflect my own work and understanding.